DBMS LAB — EXPERIMENT 09

Railway Booking: 5,000 Records, Single and Composite Indexes, EXPLAIN

Note: These are expected/representative outputs based on the described SQL and sample data, not screenshots from a live MySQL run. Exact formatting, timings and EXPLAIN plans can vary by MySQL version.

Output 1: Record count

Total_Bookings
5000

Train_No is 12001 to 12005 in a repeating pattern, so each train has exactly 1,000 bookings.

Output 2: Benchmark before indexing

With no index on Train_No, MySQL scans all 5,000 rows to find the 1,000 matches for Train_No = 12003.

id	select_type	table	type	possible_keys	key	rows	filtered	Extra
1	SIMPLE	Railway_Booking	ALL	NULL	NULL	5000	10.00	Using where

```
EXPLAIN ANALYZE:
-> Filter: (railway_booking.Train_No = 12003)  (cost=500 rows=500) (actual time=0.045..1.3 rows=1000 loops=1)
    -> Table scan on Railway_Booking  (cost=500 rows=5000) (actual time=0.043..1.1 rows=5000 loops=1)
```

Output 3: Single index (idx_train_no)

SHOW INDEX FROM Railway_Booking now lists three indexes:

Key_name	Non_unique	Seq_in_index	Column_name	Index_type
PRIMARY	0	1	Booking_ID	BTREE
PNR	0	1	PNR	BTREE
idx_train_no	1	1	Train_No	BTREE

The plan changes from a full table scan to an index lookup that reads only the 1,000 matching rows:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Railway_Booking	ref	idx_train_no	idx_train_no	5	const	1000	100.00	NULL

```
EXPLAIN ANALYZE:
-> Index lookup on Railway_Booking using idx_train_no (Train_No=12003)  (cost=101 rows=1000) (actual
time=0.085..1.07 rows=1000 loops=1)
```

Output 4: Composite index (idx_train_date)

For Train_No = 12003 with Journey_Date between 2026-11-05 and 2026-11-15, the composite index narrows the search to 334 rows using a range scan.

id	select_type	table	type	possible_keys	key	key_len	rows	filtered	Extra
1	SIMPLE	Railway_Booking	range	idx_train_no, idx_train_date	idx_train_date	9	334	100.00	Using index condition

```
EXPLAIN ANALYZE:
-> Index range scan on Railway_Booking using idx_train_date over (Train_No = 12003 AND '2026-11-05' <=
Journey_Date <= '2026-11-15'), with index condition: (...)  (cost=151 rows=334) (actual time=0.053..0.38
rows=334 loops=1)
```

Output 5: Single vs composite index usage

The optimizer prefers the composite index for both queries. For the Train_No-only query it uses only the first column of idx_train_date (key_len 5); for the date-range query it uses both columns (key_len 9).

Query	type	key	key_len	ref	rows	Extra
WHERE Train_No = 12003	ref	idx_train_date	5	const	1000	NULL
WHERE Train_No = 12003 AND Journey_Date BETWEEN ...	range	idx_train_date	9	NULL	334	Using index condition

Output 6: Performance comparison

Stage	Observation
BEFORE INDEX	Use the EXPLAIN ANALYZE result recorded above
SINGLE INDEX	Compare actual time and rows after idx_train_no
COMPOSITE INDEX	Compare actual time and rows after idx_train_date

Stage	Access type	Rows examined	Rows returned
Before index	Full table scan (ALL)	5000	1000
Single index	Index lookup (ref)	1000	1000
Composite index	Index range scan (range)	334	334

Timings in the plans above are only illustrative; compare the rows examined and the access type, which stay stable between runs.

Important

These are expected behaviors, not literal server-captured outputs. Row counts, timings, costs and plan wording must be checked against a live run of the exact SQL statements in your repository.